Software Requirements Specification

for

Peer-to-Peer Mentoring System

Version 1.0

Prepared by

Group Name: Team 5

Amitha Gundapaneni	**SE23UCSE021**	**se23ucse021@mahindrauniversity.edu.in**
Amrutha Varshini Sundaram	**SE23UCSE023**	**se23ucse023@mahindrauniversity.edu.in**
Amulya Ambati	**SE23UCSE024**	**se23ucse024@mahindrauniversity.edu.in**
Ayush Garg	**SE23UCSE032**	**se23ucse032@mahindrauniversity.edu.in**
Gonuguntla Rusheel	**SE23UCSE067**	**se23ucse067@mahindrauniversity.edu.in**
Gorantla Hrutika	**SE23UCSE069**	**se23ucse069@mahindrauniversity.edu.in**

Instructor: Mr. Koneru Gopala Krishna

Course: Software Engineering

Lab Section: *CSE1*

Teaching Assistant: *Sri Venkata Surya Phani Teja*

Date: 13th March 2026

Contents

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
First Draft v.1.1	Amrutha Varshini Sundaram	Draft with all the expected assumptions and details of the project. Our initial line of action.	12/03/26

1 Introduction

This document presents the Software Requirements Specification (SRS) for the **Peer-to-Peer Learning and Doubt Clarification System**, a platform designed to support collaborative learning within an academic environment. The system enables students to connect with peer mentors for academic guidance, schedule mentoring sessions, and participate in a doubt clarification forum where both peers and faculty members can respond to doubts. It is intended to clarify doubts that students might get during self-study, while working on assignments, or while preparing for exams. By integrating session scheduling, question-and-answer discussions, and a points-based participation system, the platform aims to create an interactive space that encourages peer-supported learning and improved student performance.

This section introduces the system and provides background information necessary to understand the rest of the document. It includes the document's purpose, the system's scope, relevant definitions and terminology, and references used in preparing this specification.

1.1 Document Purpose

This SRS defines the requirements for **Version 1.1 of the Peer-to-Peer Learning and Doubt Clarification System**. The document outlines the functional and non-functional requirements of the system, including user roles, system features, operational constraints, and expected interactions between users and the platform.

The purpose of this document is to provide a clear and structured description of the system requirements for all stakeholders involved in the project, including developers, project team members, and evaluators. It serves as a reference throughout the development lifecycle, guiding the design, implementation, and testing of the system while ensuring that all platform components align with the intended functionality and project objectives.

1.2 Product Scope

The project aims to develop an intelligent peer-to-peer learning platform designed to support collaborative academic assistance among students. The system will connect students with verified peer mentors and provide a structured environment for doubt resolution, mentoring sessions, and knowledge sharing. The platform will include features for session scheduling, a doubt clarification forum, learning pathways, and features to organize and manage academic queries. To improve user experience and efficiency, the system may incorporate intelligent mechanisms for mentor matching and filtering questions based on subject areas and relevance.

The primary objectives of the project include enabling efficient peer mentorship by allowing students to connect with suitable peer mentors, facilitating effective mentor-student matching, and validating and organizing academic questions to minimize duplicate or repetitive content. Additionally, the platform aims to support personalized learning by guiding students toward relevant resources, providing a learning path, discussions, or mentors based on their academic needs and interactions within the system.

1.3 Intended Audience and Document Overview

This document is intended for stakeholders involved in the development and evaluation of the Peer-to-Peer Learning and Doubt Clarification System. It is a document that will help the development team, instructors, and others interested in understanding our system to understand its functionality, requirements, and our goals for the system.

The document is organized to provide an overview of the system and its context, followed by sections of detailed description of system features, user roles, and operational requirements. Readers seeking a general understanding may refer to the introductory sections, while developers and evaluators may focus on the detailed requirement sections.

1.4 Definitions, Acronyms and Abbreviations

Abbreviation	Full Form
Admin	Administrator
COMET	Concurrent Object Modelling and Architectural Design Method
ML	Machine Learning
SRS	Software Requirements Specification
UML	Unified Modelling Language

1.5 Document Conventions

Formatting Conventions
This document follows the IEEE guidelines for SRS documentation. The text is written in Arial font with size 11 or 14 (Headings) and is formatted with single spacing and standard 1-inch margins. Section and subsection headings follow a structured numbering format. Comments or explanatory notes within the document are written in italics to distinguish them from the main content.

Naming Conventions
Terminology used throughout the document is kept consistent to ensure clarity. System components, features, and user roles such as Student, Peer Mentor, Faculty, and Administrator are referred to using the same terms throughout the document. Acronyms and abbreviations are defined in the glossary section when first introduced.

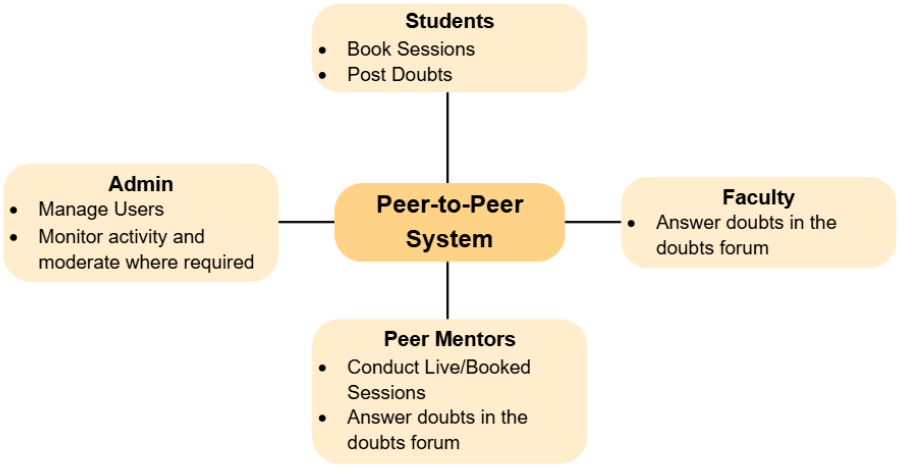
References
1. ***IEEE Computer Society.*** *IEEE Recommended Practice for Software Requirements Specifications (IEEE 830 / IEEE 29148).*
2. *Gomaa, H. Designing Concurrent, Distributed, and Real-Time Applications with UML. Addison-Wesley.*
3. *Object Management Group (OMG). Unified Modelling Language (UML) Specification.*

2 Overall Description

2.1 Product Overview

The Peer-to-Peer Learning and Doubt Clarification System is a standalone academic support platform designed to facilitate collaborative learning among students within an educational institution. The system provides a centralized environment where students can connect with peer mentors, schedule mentoring sessions, and participate in a structured forum for posting and answering academic doubts. Unlike traditional learning support systems that rely primarily on faculty assistance, this platform emphasizes peer-based guidance while still allowing faculty members to participate in discussions when necessary. It is intended to incentivize peer tutoring while also motivating students to study together through fun, interactive sessions with their cohorts.

The platform interacts with several types of users including students, peer mentors, faculty members, and administrators. Students can book mentoring sessions and post academic questions, while peer mentors and faculty members respond to doubts and provide guidance. Peer mentors can offer time slots for tutoring based on their availability or conduct live sessions for a large student audience with common requests. Administrators oversee platform activity and manage user accounts. The system operates as a web-based platform accessible through standard internet browsers, serving as an intermediary that connects users and organizes academic support resources.



2.2 Product Functionality

The SRS describes a system that provides the following major functions:

- **User Registration and Authentication** – Allow students to create accounts and securely access the platform.
- **Mentor/Faculty Profile Management** – Enable peer mentors to create and manage profiles, including their subjects of expertise and availability.
- **Session Scheduling and Booking** – Allow students to view available mentoring slots and book sessions with peer mentors.

- **Doubt Posting and Discussion Forum** – Allow students to post academic doubts and participate in discussion threads.
- **Answering and Knowledge Sharing** – Enable peer mentors and faculty members to respond to questions and provide explanations or guidance.
- **Points and Participation System** – Award points to users for answering questions or contributing useful responses in the forum. Students and Peer-Mentors are awarded points for participating in mentoring sessions.
- **Feedback and Rating System** – Allow students to rate mentoring sessions and provide feedback on peer mentors.
- **Content Organization and Search** – Enable the system to organize questions and allow users to search for existing doubts and solutions.
- **Administrative Management** – Allow administrators to manage users, update system versions, moderate discussions, and monitor platform activity.

2.3 Design and Implementation Constraints

The development of the system described in this SRS is subject to several design and implementation constraints that guide how the software must be designed and implemented.

First, the system must be designed using the **COMET method**. The COMET method provides a structured approach for object-oriented software development and supports the identification of system components, interactions, and architectural patterns during the design process. In addition, **UML** will be used as the standard modelling language to represent system structure and behavior through diagrams such as use case, class, sequence, and architectural diagrams.

The system is expected to operate as a **web-based platform** accessible through standard web browsers. Therefore, the implementation must rely on technologies compatible with modern web environments. The platform must support multiple users accessing the system simultaneously and should maintain acceptable performance levels during concurrent interactions. The system must also implement **secure authentication and data protection mechanisms** to safeguard user accounts and academic discussions.

Finally, the system should follow **standard software engineering practices and modular design principles** to ensure maintainability and future extensibility. This will allow the system to be updated or expanded as additional features are introduced. It should be easily scalable to different academic institution structures and audience sizes.

2.4 Assumptions and Dependencies

The requirements specified in this SRS are based on several assumptions regarding the operating environment and system usage. If these assumptions change, the design or implementation of the system may need to be revised.

- **Internet Availability** – It is assumed that users will have reliable internet access to interact with the web-based platform.
- **User Participation** – The effectiveness of the system depends on active participation from peer mentors and faculty members to respond to posted doubts.

- **Web-Based Environment** – The system is assumed to operate as a browser-accessible web application and will rely on standard web technologies.
- **Database Availability** – The platform assumes the availability of a backend database to store user accounts, session schedules, questions, answers, and participation points.
- **Institutional Context** – It is assumed that the system will be used within an academic environment where students and peer mentors belong to the same institution.
- **External Tools and Libraries** – The system may rely on third-party libraries, frameworks, or APIs for features such as authentication, scheduling, or communication.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system described in this SRS provides a web-based user interface that allows users to interact with the platform through standard web browsers. The interface will support interaction with common input devices, such as a keyboard, mouse, or touch-enabled screens. Users will access the system through a login page and will be presented with different dashboard views depending on their role (Student, Peer Mentor, Faculty, or Administrator).

The main interface for students will include options to post academic doubts, browse existing discussion threads, book sessions, view upcoming appointments and live sessions. Peer mentors will have access to additional features, including sharing their availability, hosting live sessions, chatting with mentees for session mode discussions, and responding to student questions. Faculty members will be able to view and contribute to discussion threads, while administrators will have access to management tools to moderate discussions and manage user accounts.

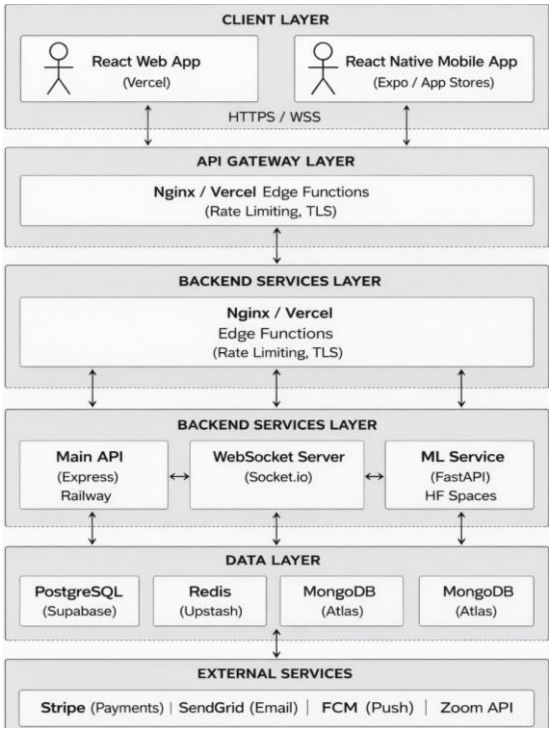
The interface will use a menu-based navigation structure with clearly labelled sections such as **Dashboard**, **Post a Doubt**, **Browse Discussions**, **Book a Session**, and **Profile**. Users will interact with the system through buttons, forms, and discussion threads displayed on the platform.

3.1.2 Hardware Interfaces

The system described in this SRS is a web-based application and does not require specialized hardware components. Users interact with the platform through standard computing devices capable of accessing the internet and running modern web browsers.
The primary hardware interfaces for the system include:

- ***Personal Computers and Laptops*** *– Users can access the platform through desktop or laptop computers using a keyboard, mouse, and web browser interface.*
- ***Mobile Devices*** *– Smartphones and tablets may also be used to access the system through mobile web browsers or touch-based interfaces.*
- ***Server Hardware*** *– The system will operate on server infrastructure that hosts the application, manages user requests, and stores system data through a database.*

These hardware components allow users to interact with the system for posting doubts, booking mentoring sessions, responding to questions, and managing platform activities. Our platform doesn't involve the use of sensors or actuators for recording and responding on data.



System Components Overview

3.1.3 Software Interfaces

The system described in this SRS interacts with several supporting software components required for its operation. These interfaces enable the platform to store data, manage user authentication, and provide access through web browsers.
The primary software interfaces include:

- **Web Browsers** – The system is accessed through standard web browsers such as *Google Chrome, Mozilla Firefox, or Safari*. These browsers provide the interface through which users interact with the platform.
- **Database Management System (DBMS)** – The system interfaces with a backend database to store and manage user information, mentoring session schedules, posted doubts, responses, participation points, and feedback data. This system uses a database hosted on *Supabase*, an open-source service that allows us to set up a real-time database compatible with web applications deployed on servers.
- **Web Server / Application Server** – The platform communicates with a server that processes user requests, manages application logic, and retrieves or updates information stored in the database. The web application will be set up using an open-source software called *Vercel*. The hosting server will be able to support a large student base accessing the system application.
- **Optional External Services** – The system may integrate with external services such as email notification systems or authentication services to support features like account verification, session reminders, or system alerts. It will also involve the services of a payment gateway if mentors wish to direct their payments through the application.

3.2 Functional Requirements

3.2.1 F1: User Registration and Authentication
The system shall allow users to create accounts and securely log in to the platform using their registered credentials. The system shall support different user roles, including Student, Peer Mentor, Faculty, and Administrator.

3.2.2 F2: User Profile Management
The system shall allow users to create and update personal profiles containing information such as name, subjects of interest or expertise, and availability.

3.2.3 F3: Mentor Availability Management
The system shall allow peer mentors to specify and update their available time slots for mentoring sessions.

3.2.4 F4: Session Booking
The system shall allow students to view available mentor time slots and book mentoring sessions.

3.2.5 F5: Doubt Posting
The system shall allow students to post academic doubts or questions within the discussion forum.

3.2.6 F6: Viewing and Searching Doubts
The system shall allow users to browse and search existing discussion threads to view previously asked questions and their answers.

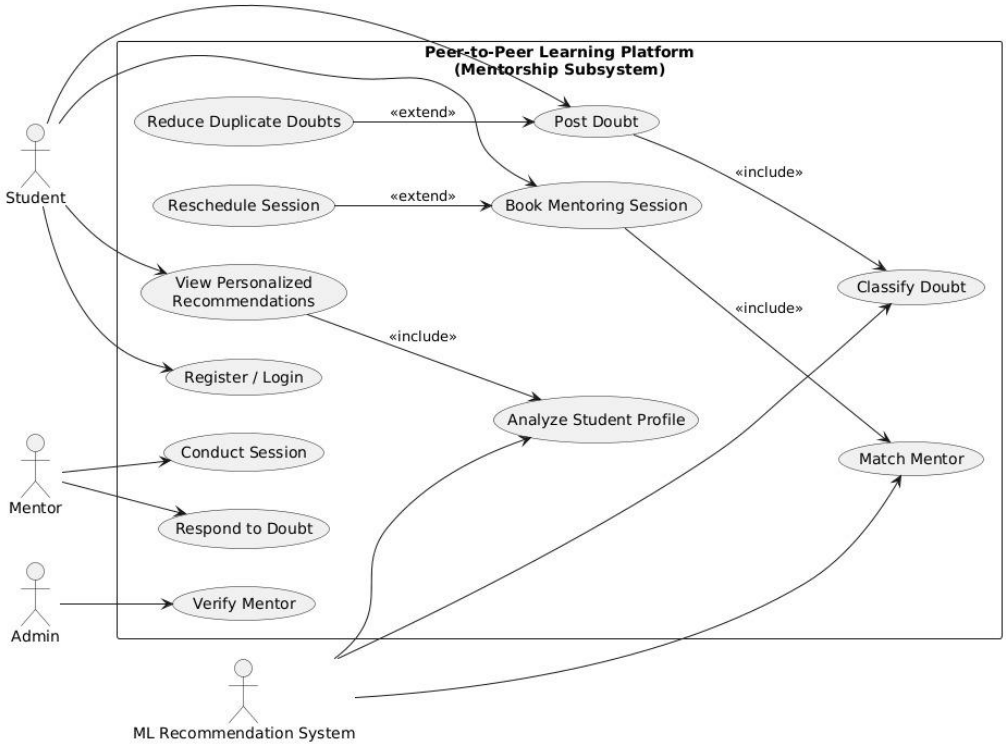
3.2.7 F7: Answering Questions
The system shall allow fellow students, peer mentors and faculty members to respond to questions posted by users.

3.2.8 F8: Participation Points System
The system shall award participation points to users who provide helpful responses to questions in the discussion forum and to students and mentors participating in sessions.

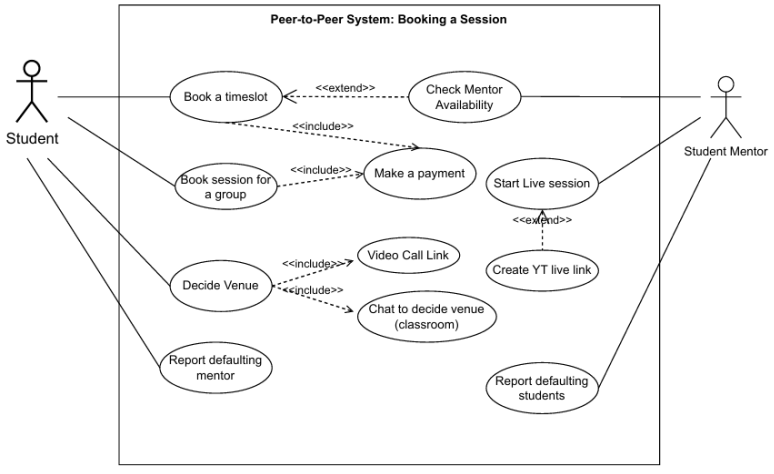
3.2.9 F9: Feedback and Rating
The system shall allow students to rate mentoring sessions and provide feedback on peer mentors.

3.2.10 F10: Administrative Management
The system shall allow administrators to manage user accounts, monitor system activity, and moderate discussion content when necessary.

3.3 Use Case Model



3.3.1 Use Case #1 (Booking a session- USE1)



Author – Amrutha Varshini Sundaram

Purpose – Maps how a student can book a peer mentoring session

Requirements Traceability –

Functional:

- View mentor profiles and availability.
- Book mentoring sessions for a selected timeslot.
- Book individual or group sessions.
- Make payments for sessions.
- Report defaulting mentors or students.

Non-Functional:

- Secure authentication and user data protection.
- System should provide high availability during booking and session times.
- Real-time communication for online sessions and fast response.
- The interface should be simple and user-friendly for students and mentors.
- Data consistency for bookings and payments.

Priority – High Priority- one of the main functions of the platform

Preconditions – A student must be registered and logged onto the platform; Stable internet connectivity

Post conditions – Both actors get booking confirmations

Actors – Student (Primary), Peer/Student Mentors

Extends – Mentor availability

Flow of Events

1. Basic Flow - The student logs into the system, selects a mentor, checks availability, and books a preferred timeslot for a session. The student completes payment, the session time is decided, and the mentor starts the session at the scheduled time.

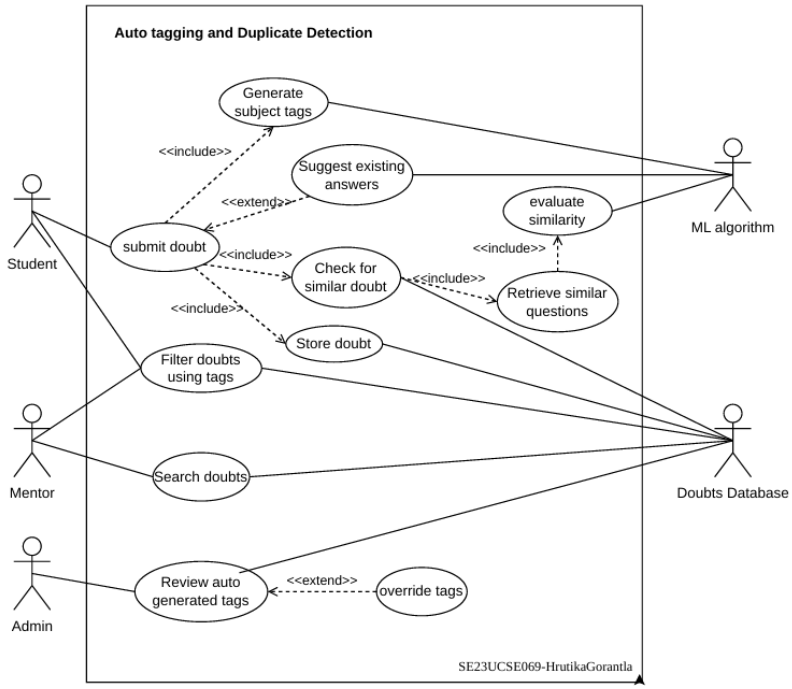
2. Alternative Flow - The student chooses to book a **group session** instead of an individual session.

3. Exceptions - If the mentor is not available for the selected timeslot, the system notifies the student. The student can book a session at a different time slot.

Includes – Making a payment for the session; Chat to decide the venue

Notes/Issues – A payment gateway needs to be set up; Group sessions may have capacity limits; Alerts have to be sent for reminders/confirmations

3.3.2 Use Case #2 (Auto Tagging and Duplicate Detection- USE2)



Author – Hrutika Gorantla

Purpose - To categorize submitted doubts automatically by matching them to similar existing questions in the database and using an ML model to generate subject tags. This helps reduce redundancy in questions while also providing previous answers if a similar doubt has already been asked.

Requirements Traceability –

Functional:

-Doubt submission
-ML-based tag generation
-Duplicate detection and suggestion of existing answers
-Search and filtering using tags
-Ability for an admin to override tags

Non-functional:

-Accurate tag generation
-Quick retrieval of similar questions

Priority - Medium. Since users can manually tag their doubts if necessary, developing an ML algorithm to auto-tag doubts is not critical. However, ML tag generation helps organize doubts more effectively, and duplicate detection can provide quick answers to users if a similar question already exists.

Preconditions - A logged-in student account must submit a doubt.

Post conditions - The generated tags should be stored in the database. Paths to similar questions should also be stored so that once one similar question is found, other related questions can be accessed through the stored path. A similarity index can then be used to verify whether the doubts accessible through that path are actually similar.

Actors –

Student – submits doubts and may answer doubts
ML Algorithm – generates tags and checks similarity
Doubts Database – stores and retrieves doubts
Mentor – answers doubts
Admin – monitors and overrides tags

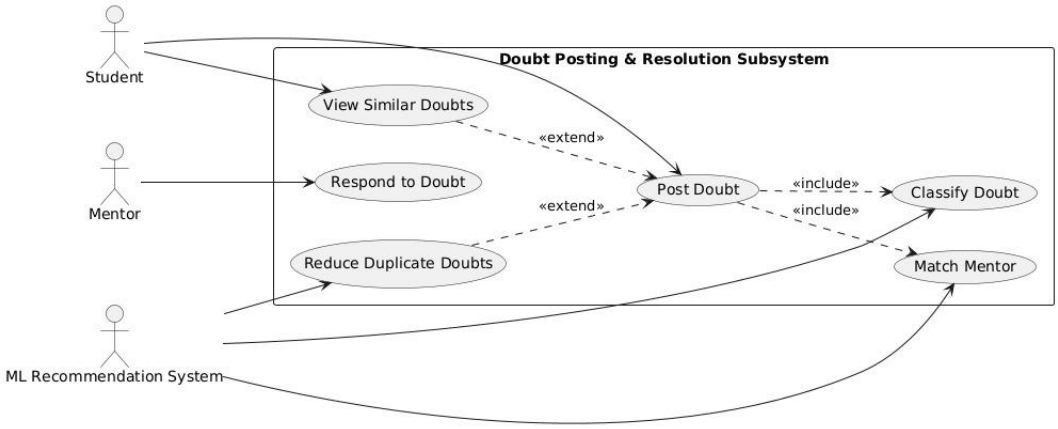
Extends –

-Suggest existing answers extend Submit doubt (suggestions appear only if similar questions are found).
-Override tags extend Review auto-generated tags (admin overrides tags only if the ML algorithm incorrectly tags something).

Flow of Events

1. Basic Flow –
   -Student submits a doubt with text and subject.
   -Platform UI sends the doubt to the ML Algorithm.
   -The ML Algorithm generates subject tags for the doubt.
   -The ML Algorithm queries the Doubts Database to check for similar doubts.
   -No duplicates are found; the generated tags are returned to the Platform UI.
   -The Platform UI stores the doubt and tags in the Doubts Database.
   -Confirmation is shown to the student.
   -Mentors are notified of the new doubt, filtered by tag.
2. Alternative Flow –
   -If similar doubts are found, the ML Algorithm returns matching questions.
   -The Platform UI displays similar doubts to the student.
   -The student may choose to view the existing answers or proceed with posting the doubt as a new question
3. Exceptions –
   -ML Algorithm unavailable: The doubt is stored without tags; manual tagging can be performed by the student or an admin.
   -Database connection failure: The submission fails, and the student is shown an error message and prompted to retry.

3.3.3 Use Case #3 (Doubt Posting and Resolution System – USE3)



Author – Ayush Garg

Purpose - Allows students to post doubts, find similar questions, and receive responses from mentors with the help of system recommendations.

Requirements Traceability –

Functional:
- Post doubts.
- View similar doubts before posting.
- Classify doubts automatically.
- Mentors shall be able to respond to posted doubts.
- Reduce duplicate doubts using ML recommendations.

Non-Functional:
- Fast search and recommendation results.
- High availability for doubt posting and responses.
- Data accuracy in doubt classification and matching.

Priority – High- Core functionality for enabling student learning and mentor interaction.

Preconditions - The student or mentor must be logged into the platform; The doubt resolution system must be operational with ML recommendation support.

Post conditions – Doubt successfully posted; System updates the doubt database and mentor interaction records.

Actors – Student; Mentor (Peer or Faculty); ML Recommender System

Extends – View Similar Doubts, Respond to Doubt, Reduce Duplicate Doubts

Flow of Events

1. Basic Flow - A student posts a doubt, the system classifies it and matches it with a suitable mentor who can respond to the doubt.

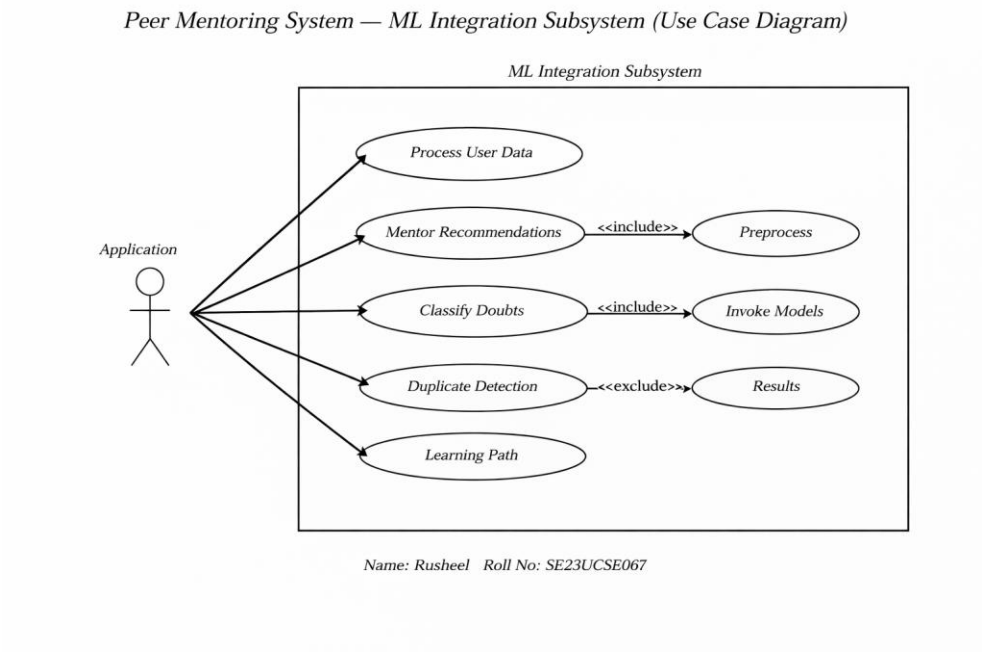
2. Alternative Flow - Before posting, the student views similar doubts suggested by the system and may choose an existing discussion instead.

3. Exceptions - The system fails to classify the doubt or find a suitable mentor, requiring manual mentor assignment.

Includes – Classify doubts; Match Mentor

Notes/Issues – Requires continuous model improvement

3.3.4 Use Case #4 (ML Integration System – USE4)



Author – Rusheel Gonuguntla

Purpose - Processes user data using machine learning to classify doubts, detect duplicates, recommend mentors, and generate personalized learning paths.

Requirements Traceability –

Functional:

- Process user data for analysis and provide mentor recommendations using ML models.
- Classify student doubts automatically.
- Detect duplicate doubts to reduce repetition.
- Generate personalized learning paths for students.

Non-Functional:

- High accuracy in classification and recommendations.
- The system should support scalability for large volumes of user data.

Priority - Medium – Supports intelligent features and improves efficiency of the mentoring platform

Preconditions - The application must be connected to the ML subsystem; User data must be available and pre-processed for analysis.

Post conditions - ML models generate predictions, classifications, or recommendations; Results are returned to the application subsystem.

Actors – Application System

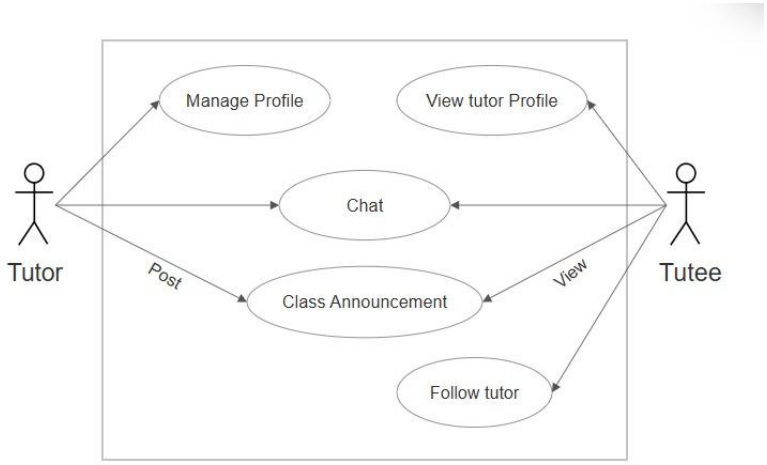
Flow of Events

1. Basic Flow - The application sends user data to the ML subsystem, which processes the data, invokes ML models, and returns predictions or recommendations.
2. Alternative Flow - The subsystem preprocesses the data and runs specific ML tasks such as mentor recommendation, doubt classification, or learning path generation.
3. Exceptions - If ML models fail or data is insufficient, the system returns an error or fallback recommendation.

Includes – Preprocess; Invoke Models

Notes/Issues – Depends on training and quality of data.

3.3.5 Use Case #5 (Messaging Forum – USE5)



Author – Amulya Ambati

Purpose - Allows tutors and mentees to communicate, manage profiles, post class announcements, and follow tutors within the mentoring platform.

Requirements Traceability –
Functional:

- Allow tutors to manage their profiles.
- Allow mentees to view tutor profiles.
- Tutors and mentees can communicate through chat.
- Tutors can post class announcements.
- Allow mentees to view announcements posted by tutors.
- The system shall allow mentors to follow tutors for updates.

Non-Functional:

- Support real-time messaging between users.
- System should ensure secure communication and user privacy.
- Provide fast loading of profiles and announcements.

Priority - High – Essential for communication between tutors and mentees.

Preconditions - The tutor or mentees must be registered and logged into the system; The messaging/forum system must be active and accessible.

Post conditions - Messages or announcements are successfully delivered or posted; Tutor profiles or follow relationships are updated in the system.

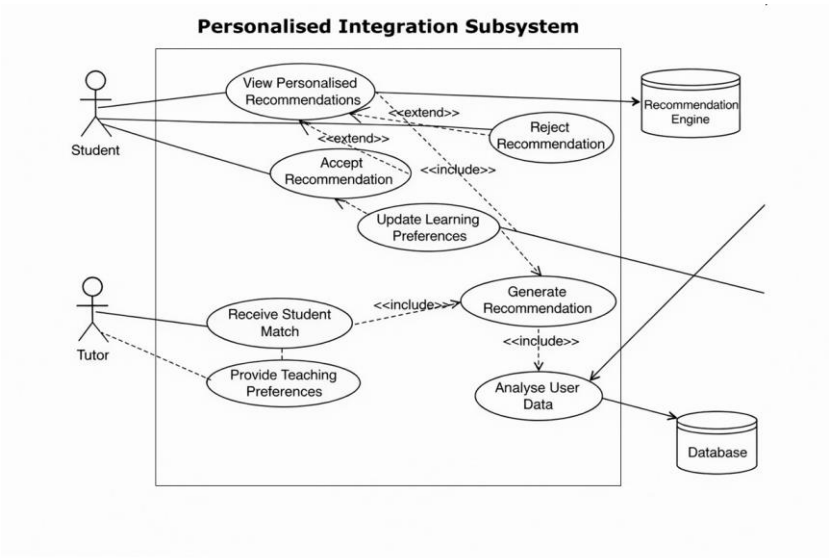
Actors – Tutors, peer-mentors, faculty

Flow of Events

1. Basic Flow - A tutor or mentee logs into the system, views profiles or announcements, and communicates using the chat feature.
2. Alternative Flow - A tutor posts a class announcement or updates their profile, while a mentee chooses to follow a tutor for updates.
3. Exceptions - If messaging fails or the profile cannot be loaded, the system notifies the user and requests retry.

Includes – Manage Profile; View Tutor Profile; Chat; Class Announcement; Follow Tutor

3.3.6 Use Case #6 (Personalised Recommendation System – USE6)



Author – Amitha Gundapaneni

Purpose - Provides personalized mentor and learning recommendations to students by analyzing user data and updating learning preferences.

Requirements Traceability –

Functional:

- Allow students to view, accept, or reject personalized recommendations.
- Update learning preferences based on user actions.
- Generate mentor or learning recommendations.
- System shall match students with suitable tutors.

Non-Functional:

- Support fast processing and high accuracy of recommendation requests.
- Ensure secure storage and handling of user data.

Priority – Low- Important for improving learning efficiency and user experience.

Preconditions - The student or tutor must be logged into the system; User data and learning preferences must be stored in the database.

Post conditions - Personalized recommendations are generated and displayed to the student; Updated learning preferences are stored in the system database.

Actors – Student; Tutor; Recommendation Engine; Database

Extends – Accept/ Reject recommendation

Flow of Events

1. Basic Flow - The student views personalized recommendations generated by the system and may accept them, which updates learning preferences and mentor matches.

2. Alternative Flow - The student rejects a recommendation, prompting the system to update preferences and generate alternative recommendations.

3. Exceptions - If recommendation generation fails or user data is unavailable, the system notifies the user and retries the recommendation process.

Includes – Update Learning Preferences; Generate Recommendation; Analyse User Data; Receive Student Match

Notes/Issues - Recommendation accuracy depends on quality of collected user data; Continuous updates to learning preferences may be required to improve recommendation relevance.

4 Other Non-functional Requirements

4.1 Performance Requirements

P1. System Response Time
The system shall respond to user requests such as loading dashboards, viewing discussion threads, or submitting questions within **5 seconds** under normal operating conditions.

P2. Session Booking Processing
The system shall process and confirm mentoring session bookings within **10 seconds** after a student submits a booking request.

P3. Concurrent User Support
The system shall support **multiple users accessing the platform simultaneously** without a significant weakening in performance.

P4. Discussion Posting Time
The system shall allow users to post questions or responses in the discussion forum and display the updated thread within **5 seconds**.

P5. Search and Retrieval Performance
The system shall return results for doubt searches or discussion queries within **3 seconds** for typical database sizes.

4.2 Safety and Security Requirements

S1. User Authentication
The system shall require users to authenticate using valid login credentials before accessing platform features such as posting doubts, booking sessions, or responding to discussions.

S2. Role-Based Access Control
The system shall enforce role-based access permissions so that users can only access functions appropriate to their role (Student, Peer Mentor, Faculty, or Administrator).

S3. Secure Communication
All communication between users and the platform shall be transmitted using secure internet protocols (e.g., HTTPS) to protect user data and login credentials.

S4. Data Privacy Protection
The system shall protect personal user information and ensure that user data is stored securely and accessed only by authorized users.

S5. Content Moderation and Misuse Prevention
The system shall allow administrators to monitor and moderate discussions to prevent misuse, inappropriate content, or harmful interactions between users.

S6. Session Integrity
The system shall ensure that mentoring session bookings and discussion records are accurately stored and protected from unauthorized modification.

4.3 Software Quality Attributes

4.3.1 Reliability

The system shall operate reliably during normal use and maintain consistent access to core platform features such as posting doubts, viewing discussions, and booking mentoring sessions. The system shall minimize failures during user interactions and ensure that user actions such as posting questions, booking sessions, or submitting responses are properly stored in the database.

To support reliability, the system will maintain persistent data storage for user accounts, discussion threads, and session bookings. Error handling mechanisms shall be implemented to manage unexpected system failures and prevent data loss during system operations. Policies will be implemented to validate database data and maintain data consistency.

4.3.2 Usability

The system shall provide an intuitive and user-friendly interface that allows students, peer mentors, faculty, and administrators to navigate the platform with minimal training. The interface shall use clear navigation menus, labelled actions, and consistent layout structures to help users easily access features such as posting doubts, browsing discussions, and scheduling mentoring sessions.

The system shall prioritize **ease of use over complexity**, ensuring that users can perform common tasks such as submitting questions or booking sessions in a small number of steps. The system shall provide the users with dark and light modes to access the application according to their system preferences. Profile icons will also be included to add a personal touch to individual profiles.

4.3.3 Maintainability

The system shall be designed using modular software architecture so that individual components, such as user management, session scheduling, and discussion forums, can be updated or modified independently.

To support maintainability, the development process will follow structured design methods such as **COMET**, and system models will be documented using **UML diagrams**. Clear documentation and consistent coding practices will allow future developers to maintain or extend the system with minimal difficulty.

4.3.4 Flexibility and Adaptability

The system shall be designed to support future enhancements and changes in system requirements. The modular design of the platform will allow additional features such as new discussion categories, expanded mentor matching methods, or additional user roles to be incorporated without major redesign of the existing system.

The architecture will also allow the platform to adapt to different academic environments or institutions with minimal configuration changes. The web-server platform is receptive to updates in application functions and software updation.

5 Other Requirements

5.1 Database Requirements

The system shall maintain a structured database to store information related to user accounts, mentoring session schedules, discussion threads, posted questions, responses, participation points, and feedback data. The database shall ensure data consistency and support efficient retrieval of information such as discussion posts and mentor availability.
The database design shall support multiple concurrent users accessing and updating data while maintaining data integrity.

5.2 Logging and Monitoring

The system shall maintain logs of important system activities such as user logins, session bookings, and posted discussions. These logs will help administrators monitor system usage, identify potential issues, and maintain system reliability.
Administrative users shall be able to review logs when necessary to investigate errors or misuse of the platform.

5.3 Scalability

The system architecture shall support future expansion to accommodate increasing numbers of users, discussion posts, and mentoring sessions. The platform should be designed so that additional system resources, such as server capacity or database storage, can be integrated without major changes to the system design.

Appendix A – Data Dictionary

Data Item	Type	Description	Possible Values / Format	Related Operations
StudentID	Identifier	Unique ID assigned to every user in the system	Integer / UUID	Login, Profile Management
UserRole	State Variable	Defines the role of the user	Student, Tutor, Mentor	Access Control, Matching
UserProfile	Data Structure	Stores personal and academic information of users	Name, skills, subjects	Manage Profile
DoubtID	Identifier	Unique ID for each posted doubt	Integer	Doubt Posting
DoubtText	Input	The question or doubt submitted by a student	Text/String	Post Doubt, Classification
DoubtCategory	State Variable	Category assigned to a doubt by ML model	Subject tags	Classify Doubts
SimilarDoubts	Output	List of related doubts detected by the system	List of DoubtIDs	Duplicate Detection
MentorID	Identifier	Unique ID for each mentor	Integer	Mentor Matching
MentorRecommendation	Output	Suggested mentor for a student doubt	MentorID	Mentor Recommendation
LearningPreference	State Variable	Student learning interests and preferred topics	Subjects / skills	Personalization
ChatMessage	Input/Output	Message exchanged between tutor and mentee	Text	Messaging System
Announcement	Input/Output	Class updates posted by tutors	Text	Class Announcements
SessionID	Identifier	Unique ID for mentoring session	Integer	Session Booking
SessionLink	Output	Generated link for online session	URL	Video Session
MLModelResult	Output	Result returned by ML model prediction	Category / Score	ML Processing
UserData	Input	Data used by ML models for analysis	Interaction logs	ML Integration
DatabaseRecord	Data Storage	Stored system information	Structured records	Data Retrieval

Appendix B - Group Log

Date: 06/02/2026
Attendees: All team members
Key Discussions/Decisions:

- Brainstorming and ideation of core platform features for the Peer Mentoring System.
- Identified the need for **separate authentication flows and dashboards** for two primary user roles: **students and mentors**.
- Proposed a **user questionnaire during initial login** to collect user preferences and background information, which will be used by the ML subsystem to generate a **personalized learning path**.
- Designed the concept of a **Sessions Board**, which will display both **past and upcoming mentoring sessions**, along with functionality for students to **schedule sessions with mentors**.
- Designed a **Doubt Board** to allow students to post questions and track previously asked doubts. The board will display both **active and resolved doubts**.
- Identified machine learning components required for the system:
    - **Mentor Recommendation Model:** Suggest mentors based on questionnaire responses and user preferences.
    - **Doubt Classification System:** Automatically tag doubts with relevant topics.
    - **Duplicate Doubt Detection:** Detect previously asked questions to reduce redundancy.
    - **Adaptive Learning Path Model:** Generate and dynamically adjust personalized learning paths based on user interaction and progress.

Action Items:

- Create **frontend mockups and UI wireframes** for the proposed system.
- Identify and begin learning the **required programming languages, frameworks, and tools** for implementation.
- Draft the **Statement of Work (SOW)** outlining project scope and responsibilities.

Date: 15/02/2026
Attendees: All team members
Key Discussions/Decisions:

- Conducted a **review and evaluation of the initial UI design mock-ups** created for the system.
- Selected the **most suitable interface design** based on usability, layout clarity, and feature accessibility.
- Defined and structured the **major subsystems** of the application architecture.
- Assigned **individual subsystem responsibilities** to each team member to distribute development tasks.

Action Items:

- Further refine and elaborate the **selected website mock-up and interface design**.
- Begin creating **detailed use case specifications** for each subsystem.

Date: 28/02/2026
Attendees: All team members
Key Discussions/Decisions:

- Reviewed a **functional prototype/demo version of the website**, analysing UI behaviour and workflow.
- Identified necessary **design improvements and feature adjustments**.
- Discussed backend architecture and **divided backend development tasks** among team members, including:

    - Doubt management system
    - Session scheduling system
    - Database design and management
    - Authentication and user management
- Successfully **configured the team's GitHub repository** for collaborative development and version control.

Action Items:

- Begin **backend implementation** for the assigned subsystems.
- Implement the **design modifications identified during the review**.

Date: 09/03/2026
Attendees: All team members
Key Discussions/Decisions:

- Reviewed **database schemas and tables implemented using Supabase**.
- Evaluated an **experimental setup for database interaction**, including retrieving and posting data using **Postman API requests**.
- Reviewed and discussed **updated design iterations** and finalized ideas for the **final UI mock-up design**.
- Decided to begin **frontend development using React** once the final design is confirmed.
- Reviewed the **user authentication system**, including:
    - Login API implementation
    - JWT-based authentication
    - Authentication middleware for protected routes
- Organized and discussed the **structure and required sections for the Software Requirements Specification (SRS) document**.

Action Items:

- Integrate and **combine individual backend modules** to ensure proper system functionality.
- Implement the **final design updates** across the application interface.
- Begin **writing and compiling the SRS document**.

*other real-time communication through a team WhatsApp group for any doubts, suggestions, and work progress updates